TEMA 05

Funciones en PHP

Desarrollo Web en Entorno Servidor
Tema 5 · Curso 2026/2027

Conceptos · Código · Ejemplos · Actividades

Qué aprenderás

- Definir funciones y llamarlas con los argumentos adecuados.
- Separar los cálculos de la presentación de sus resultados.
- Usar parámetros, valores por defecto y declaraciones de tipo.
- Distinguir paso por valor, paso por referencia y ámbito de las variables.
- Devolver resultados y organizar funciones en varios archivos.

Requisitos: condiciones, bucles y arrays. Los ejemplos son independientes salvo donde se indican archivos relacionados.

Qué aporta una función

- Agrupa instrucciones para resolver una tarea concreta.
- Puede recibir datos de entrada mediante parámetros.
- Puede devolver un resultado mediante return.
- Permite reutilizar la misma operación y comprobarla con distintos datos.

Ejemplo: calcular un descuento una sola vez y llamar a la función para cada producto.

Definir y llamar a una función

```
function saludar(): void {
    echo "Hola, clase<br>";
}

saludar();
saludar();
```

- Definir la función no ejecuta su cuerpo. Cada llamada lo ejecuta.
- void indica que la función no devuelve un valor de resultado.

Escribimos function en minúsculas y usamos nombres descriptivos. El ejemplo muestra el saludo dos veces.

Referencia [1, 2] · enlaces al final

Una cuenta atrás reutilizable

```
function cuentaAtras(int $inicio): void {
    for ($i = $inicio; $i >= 1; $i--) {
        echo $i . " ";
    }
    echo "¡Fin!<br>";
}

cuentaAtras(3);
cuentaAtras(5);
```

Salida por líneas: 3 2 1 ¡Fin! / 5 4 3 2 1 ¡Fin! $inicio es un parámetro; 3 y 5 son los argumentos de las llamadas.

Parámetros y resultado

```
function sumar(int $a, int $b): int {
    return $a + $b;
}

$total = sumar(4, 6);
echo $total; // 10
```

- int $a e int $b describen las entradas; : int describe el resultado.

return entrega el resultado al código que hizo la llamada. Ese código decide si lo muestra, lo guarda o lo utiliza en otro cálculo.

Referencia [2] · enlaces al final

echo y return tienen funciones distintas

```php
function doble(int $numero): int {
    return $numero * 2;
}

$resultado = doble(4); // No imprime nada.
echo $resultado;      // 8
echo doble(5) + 1;    // 11
```

- Sin separador, los dos echo consecutivos muestran 811.

Una función que imprime no equivale a una que devuelve un valor. Para cálculos reutilizables, suele ser mejor devolver el resultado.

return termina la función

```php
function dividir(float $a, float $b): ?float {
    if ($b == 0.0) {
        return null;
    }
    return $a / $b;
}

var_dump(dividir(10, 2)); // float(5)
var_dump(dividir(10, 0)); // NULL
```

?float permite devolver float o null. En este ejemplo null indica que no se puede dividir; el código que llama debe comprobarlo.

Referencia [2] · enlaces al final

Valores por defecto

```php
function saludar(string $nombre = "clase"): string {
    return "Hola, " . $nombre;
}

echo saludar();       // Hola, clase
echo "<br>";
echo saludar("Ana");  // Hola, Ana
```

- Se usa "clase" solo cuando no se proporciona ese argumento.

Un valor por defecto hace opcional el argumento. No es una tercera forma de paso de parámetros.

Referencia [1] · enlaces al final

Parámetros obligatorios y opcionales

```php
function aplicarIva(float $base, float $tipo = 0.21): float {
    return $base * (1 + $tipo);
}

echo aplicarIva(100);       // 121
echo "<br>";
echo aplicarIva(100, 0.10); // 110
```

Coloca primero los parámetros obligatorios. Supuestos: base y tipo no negativos. Para dinero real, define una política de precisión y redondeo.

Referencia [1] · enlaces al final

Paso por valor

```php
function incrementar(int $numero): int {
    $numero++;
    return $numero;
}

$original = 4;
$nuevo = incrementar($original);
echo $original . " / " . $nuevo; // 4 / 5
```

Con escalares y arrays, modificar el parámetro por valor no modifica la variable original. Los objetos tienen un comportamiento que se estudiará aparte.

Referencia [1] · enlaces al final

Paso por referencia

```
function incrementar(int &$numero): void {
    $numero++;
}

$contador = 4;
incrementar($contador);
echo $contador; // 5
```

- & aparece en la declaración del parámetro, no en la llamada.

La referencia permite modificar la variable del llamador; no es un puntero que se maneje como una dirección de memoria. Debes pasar una variable.

Referencia [1] · enlaces al final

Valor, referencia y valor por defecto

Concepto	Qué significa	Ejemplo
Por valor	La asignación local no cambia el original	function f(int $x)
Por referencia	La función puede cambiar la variable externa	function f(int &$x)
Valor por defecto	Permite omitir un argumento	function f(int $x = 0)

Una referencia puede ser útil, pero devolver un resultado suele hacer más visibles los cambios.

Referencia [1] · enlaces al final

EJEMPLO RESUELTO

Tipos y modo estricto

```php
<?php
declare(strict_types=1);

function sumar(int $a, int $b): int {
    return $a + $b;
}

echo sumar(2, 3); // 5
// sumar("2", 3); produciría TypeError aquí.
```

strict_types se declara al inicio del archivo. Para argumentos escalares de estas llamadas importa el archivo que llama. No valida reglas como «nota entre 0 y 10».

Referencia [2] · enlaces al final

Ámbito local

```php
$nombre = "Ana";
function mostrarNombre(): void {
    $nombre = "Luis";
    echo $nombre . "<br>";
}

mostrarNombre();
echo $nombre;
```

- Salida en el navegador: Luis y después Ana.

Son variables distintas. Los parámetros y las variables creadas dentro de una función tienen ámbito local.

Referencia [3] · enlaces al final

Variables globales: acceso explícito

```php
$contador = 0;
function registrarVisita(): void {
    global $contador;
    $contador++;
}
registrarVisita();
echo $contador;            // 1
echo $GLOBALS["contador"]; // 1
```

Una variable externa no entra automáticamente en el ámbito de la función. global y $GLOBALS permiten acceder a ella, pero crean dependencias menos visibles.

Referencia [3] · enlaces al final

Datos de entrada explícitos

```php
function incrementarVisitas(int $visitas): int {
    return $visitas + 1;
}

$visitas = 0;
$visitas = incrementarVisitas($visitas);
echo $visitas; // 1
```

- La función recibe lo que necesita y devuelve el nuevo valor.

Esta versión se puede probar sin preparar una variable global con un nombre concreto.

Una variable local static

```php
function siguienteNumero(): int {
    static $numero = 0;
    $numero++;
    return $numero;
}

echo siguienteNumero() . " "; // 1
echo siguienteNumero() . " "; // 2
echo siguienteNumero();       // 3
```

static conserva el valor entre llamadas en la misma ejecución. En una petición web convencional no sustituye a una sesión ni a una base de datos.

Referencia [3] · enlaces al final

Devolver varios datos en un array

```php
function resumen(array $notas): array {
    $cantidad = count($notas);
    $suma = array_sum($notas);
    $media = $cantidad > 0 ? $suma / $cantidad : null;
    return ["cantidad" => $cantidad, "media" => $media];
}

$datos = resumen([6, 8, 10]);
echo $datos["cantidad"] . " / " . $datos["media"];
// 3 / 8
```

return devuelve un único valor: aquí es un array con dos campos. Supuesto: las notas son numéricas. Para [] se devuelve cantidad 0 y media null.

Ejemplo: el mayor de tres números

```php
function mayorDeTres(int $a, int $b, int $c): int {
    $mayor = $a;
    if ($b > $mayor) {
        $mayor = $b;
    }
    if ($c > $mayor) {
        $mayor = $c;
    }
    return $mayor;
}
echo mayorDeTres(17, 8, 10); // 17
```

También existe max($a, $b, $c). Implementarlo aquí sirve para practicar parámetros, condiciones y return.

Funciones en otro archivo

```php
<?php
// funciones.php
declare(strict_types=1);

function sumar(int $a, int $b): int {
    return $a + $b;
}

function esPar(int $numero): bool {
    return $numero % 2 === 0;
}
```

Un archivo que solo contiene PHP puede omitir la etiqueta de cierre. Evita ejecutar salidas al cargar una biblioteca de funciones.

Cargar funciones con require_once

```php
<?php
// principal.php
declare(strict_types=1);
require_once __DIR__ . "/funciones.php";

echo sumar(2, 15); // 17
echo "<br>";
echo esPar(8) ? "Par" : "Impar"; // Par
```

- Guarda ambos archivos en la misma carpeta y ejecuta principal.php.

require_once es una construcción del lenguaje. __DIR__ parte de la carpeta del archivo actual; _once evita cargar el mismo archivo de nuevo.

Referencia [4] · enlaces al final

include y require

Construcción	Si no se encuentra el archivo	Carga repetida
include	Emite un aviso; puede continuar	Sí
require	Lanza Error; sin capturarlo, termina	Sí
include_once	Como include	Evita cargarlo otra vez
require_once	Como require	Evita cargarlo otra vez

Para funciones imprescindibles usa require_once. La ruta debe estar controlada por la aplicación, no construida directamente desde una entrada del usuario.

Referencia [4] · enlaces al final

Ampliación: argumentos con nombre

```
function precioFinal(float $base, float $descuento = 0): float {
    return $base * (1 - $descuento / 100);
}

echo precioFinal(descuento: 10, base: 100); // 90
```

- Desde PHP 8 puedes indicar el nombre de cada parámetro.
- Ayuda a identificar el significado de los argumentos.

El nombre debe coincidir con la declaración. Si mezclas argumentos posicionales y nombrados, los posicionales van primero.

Referencia [1] · enlaces al final

Ampliación: una función flecha

```php
$doble = fn(int $n): int => $n * 2;
echo $doble(4); // 8

$numeros = [1, 2, 3];
$dobles = array_map($doble, $numeros);
// $dobles: [2, 4, 6]
```

- fn define una función anónima de una expresión; su resultado se devuelve.

Esta ampliación sirve de introducción a callbacks: funciones que se pasan a otra función, como array_map.

Referencia [5] · enlaces al final

ACTIVIDAD

Práctica: funciones para trabajar con notas

- Crea esAprobado(float $nota): bool. Supón que recibe una nota entre 0 y 10.
- Crea media(array $notas): ?float. Devuelve null si el array está vacío.
- Crea contarAprobados(array $notas): int utilizando esAprobado.
- Guarda las funciones en funciones.php y pruébalas desde principal.php con require_once.
- Con [4, 5, 9] debes obtener media 6 y dos aprobados. Prueba también [] y [5].

Las funciones deben devolver resultados; principal.php se encarga de mostrarlos. No uses variables globales.

Repaso y comprobaciones

- Parámetro: variable de la declaración. Argumento: dato de la llamada.
- return entrega un resultado y termina la función; echo escribe en la salida.
- Por referencia se puede modificar la variable externa; un valor por defecto permite omitir el argumento.
- Los tipos no comprueban por sí solos las reglas del problema.
- Prueba valores habituales, límites y entradas vacías antes de reutilizar una función.

Antes de escribir una función, concreta qué recibe, qué devuelve y qué cambios produce fuera de ella.

Referencias y alcance

[1] Parámetros y argumentos

[2] Declaraciones de tipo y strict_types

[3] Ámbito de variables y static

[4] require_once e inclusión de archivos

[5] Funciones flecha

[6] Funciones definidas por el usuario

Adaptación del material original del docente. Revisión: septiembre de 2026. Ejemplos para PHP 8.x; ampliaciones señaladas.